

Lista V3

Security Review

Cantina Solo review by:
Chinmay Farkya, Security Researcher

April 30, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	5
3.1	Informational	5
3.1.1	ListV3Factory differs from the original pattern used by Uniswap to prevent code reuse	5
3.1.2	Storage gap in ERC721PermitUpgradeable can be removed	5
3.1.3	Add License Notice	5
3.1.4	Additional documentation can be added for POOL_INIT_CODE_HASH in the README	6

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

A security review is a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While the review endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that a security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Lista DAO functions as the open-source decentralized stablecoin lending protocol powered by LSDfi.

From Apr 24th to Apr 29th the security researchers conducted a review of [lista-v3](#) on commit hash [35c9b5ab](#). A total of **4** issues were identified:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	0	0	0
Gas Optimizations	0	0	0
Informational	4	4	0
Total	4	4	0

2.1 Scope

The security review had the following components in scope for [lista-v3](#) on commit hash [35c9b5ab](#):

```
src
├── core
│   ├── ListaV3Factory.sol
│   ├── ListaV3Pool.sol
│   ├── ListaV3PoolDeployer.sol
│   ├── NoDelegateCall.sol
│   └── libraries
│       ├── BitMath.sol
│       ├── FixedPoint128.sol
│       ├── FixedPoint96.sol
│       ├── FullMath.sol
│       ├── LiquidityMath.sol
│       ├── LowGasSafeMath.sol
│       ├── Oracle.sol
│       ├── Position.sol
│       ├── SafeCast.sol
│       ├── SqrtPriceMath.sol
│       ├── SwapMath.sol
│       ├── Tick.sol
│       ├── TickBitmap.sol
│       ├── TickMath.sol
│       ├── TransferHelper.sol
│       └── UnsafeMath.sol
├── periphery
│   ├── NonfungiblePositionManager.sol
│   ├── NonfungibleTokenPositionDescriptor.sol
│   ├── SwapRouter.sol
│   ├── V3Migrator.sol
│   └── base
│       ├── BlockTimestamp.sol
│       ├── ERC721Permit.sol
│       ├── ERC721PermitUpgradeable.sol
│       ├── LiquidityManagement.sol
│       ├── Multicall.sol
│       ├── PeripheryImmutableState.sol
│       ├── PeripheryPayments.sol
│       ├── PeripheryPaymentsWithFee.sol
│       └── PeripheryValidation.sol
```

```
|   |— PoolInitializer.sol
|   |— SelfPermit.sol
|— lens
|   |— ListaInterfaceMulticall.sol
|   |— Quoter.sol
|   |— QuoterV2.sol
|   |— TickLens.sol
|— libraries
|   |— BytesLib.sol
|   |— CallbackValidation.sol
|   |— ChainId.sol
|   |— HexStrings.sol
|   |— LiquidityAmounts.sol
|   |— NFTDescriptor.sol
|   |— NFTSVG.sol
|   |— OracleLibrary.sol
|   |— Path.sol
|   |— PoolAddress.sol
|   |— PoolTicksCounter.sol
|   |— PositionKey.sol
|   |— PositionValue.sol
|   |— SqrtPriceMathPartial.sol
|   |— TokenRatioSortOrder.sol
|   |— TransferHelper.sol
```

3 Findings

3.1 Informational

3.1.1 ListaV3Factory differs from the original pattern used by Uniswap to prevent code reuse

Severity: Informational

Context: [ListaV3Factory.sol#L1](#)

Description: The `UniswapV3Factory` uses a `noDelegateCall` modifier on its `createPool` function.

The reasoning given by the Uniswap team for using `noDelegateCall` in the original Factory contract is to prevent circumventing the license, even GPL license. As pointed out [here](#). If this was not present, anyone could deploy a proxy and use the deployed factory contract as an implementation, thus allowing code reuse.

Lista has modified the factory to make it an upgradeable contract, which means `ListaV3Factory` is now an implementation contract behind a proxy and `noDelegateCall` had to be removed to make it work.

Now other proxies that want to reuse this code can easily point to this deployed `ListaV3Factory` implementation contract.

Lista will use the Factory code under a GPL License, but a third-party proxy can still use it as it wants without meeting the GPL requirements, if they want to.

This could be incompatible with GPL license compliance : The legal status of `delegatecall`-into-licensed-bytecode is itself unsettled; `NoDelegateCall` was a technical deterrent, not a settled legal interpretation.

Recommendation: Consider documenting this potential incompatibility with Uniswap's GPL interpretation, or avoid making the factory upgradeable to prevent legal issues in the future.

Lista DAO: Fixed in commit [d577c43a](#).

Chinmay: Fix verified.

3.1.2 Storage gap in ERC721PermitUpgradeable can be removed

Severity: Informational

Context: [ERC721PermitUpgradeable.sol#L27](#)

Description: The `NonFungiblePositionManager` contract has a storage gap defined at the end of its storage slots. It is the child contract, so having a storage gap at its tail makes sense. It also inherits from `ERC721PermitUpgradeable` contract, which is an abstract contract and will never exist independently. This contract also declares a storage gap. This storage gap in `ERC721PermitUpgradeable` might be unnecessary if no modifications are expected here that are independent of the actual child contract. If we need to add any new state variables to the deployed `NonFungiblePositionManager` contract in an upgrade, we can add those to its own storage gap. Right now, `ERC721PermitUpgradeable` is only used in the `NonFungiblePositionManager` contract and we only need to streamline the storage slots of these two contracts, so this change will still be upgrade-safe.

Recommendation: Consider removing the storage gap in `ERC721PermitUpgradeable.sol` if it is not intended to be used in other contracts, and will not be modified independently of this `NonFungiblePositionManager` contract. Make sure to thoroughly review the storage structure if `ERC721PermitUpgradeable` sources are ever modified.

Lista DAO: Fixed in commit [f8f4410a](#).

Chinmay: Fix verified.

3.1.3 Add License Notice

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: There needs to be a License Notice file declaring that the codebase has been forked from Uniswap V3 under the GPL License.

Such a notice already existed in the [Moolah repository](#), but might have been missed when porting the codebase to a new repository.

Recommendation: Consider adding a License notice.

Lista DAO: Fixed in commit [1a3391f1](#).

Chinmay: Fix verified.

3.1.4 Additional documentation can be added for `P00L_INIT_CODE_HASH` in the README

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: The `ListaV3Factory` uses a pre-computed constant `initCodeHash` for deploying new Lista pools, just like Uniswap does. In the project README, under "Operational Notes" it has been acknowledged that the `P00L_INIT_CODE_HASH` depends on a lot of factors even if the `ListaV3Pool` solidity code does not change.

Recommendation: Consider adding a note on re-verifying the `P00L_INIT_CODE_HASH` every time Lista plans to deploy these contracts on a new chain, as the `init_code` might change on a different chain.

Lista DAO: Fixed in commit [743d1656](#).

Chinmay: Fix verified.